

Semântica das Linguagens Funcionais

Revisões

Grupo 1

Considere os seguintes termos do lambda calculus puro:

$$\begin{array}{ll} Y & \equiv (\lambda f.(\lambda x.f(x x))(\lambda x.f(x x))) & I & \equiv (\lambda x.x) \\ F & \equiv (\lambda a.\lambda b.b) & S & \equiv (\lambda x.\lambda y.\lambda z.x z (y z)). \end{array}$$

1. Considere o termo $SI(SFx)(Iy)$, onde x e y são variáveis livres. Apresente a sequência da ordem aplicativa de redução deste termo, até à sua forma normal. Sublinhe o β -redex que é selecionado em cada passo de redução.
2. Considere agora o termo $F(YI)$. Como classifica este termo quanto à normalização? Justifique a sua resposta.

Grupo 2

Considere a linguagem de programação funcional FUNcbn que estudamos.

1. Apresente, passo a passo, a sequência da avaliação *call-by-name*, até à forma canónica, da expressão

```
let fun  $\equiv \lambda x.\lambda l.$  listcase  $l$  of ( $\langle 0, x \rangle, \lambda h.\lambda t. \langle h, x \rangle$ ),  
    ex  $\equiv (3*2)::0::\text{nil}$   
in fun (5+5) ex
```

(Obs.: Como responderia se a estratégia fosse *call-by-value*?)

2. Defina a função `contaEq`, de tipo $\text{List}(\text{Int} \times \text{Int}) \rightarrow \text{Int}$, que conta o número de pares da lista em que a primeira componente do par é igual à segunda componente.
3. Construa uma árvore de prova do seguinte juízo

$f : \text{Int} \rightarrow \text{List Int} \rightarrow \text{Int} \times \text{Int}, a : \text{Unit} + (\text{Int} \times \text{Int}) \vdash \lambda l. \text{sumcase } a \text{ of } (\lambda x. \langle 0, 0 \rangle, \lambda y. f y.1 l) : \text{List Int} \rightarrow \text{Int} \times \text{Int}$

Grupo 3

Pretende-se estender a linguagem de programação funcional FUNcbn com um novo tipo de dados para representar *árvores ternárias*, com informação nos nodos e folhas vazias. Em Haskell este tipo de dados poderia ser declarado assim

```
data TTree a = Null | Node a (TTree a) (TTree a) (TTree a)
```

1. Defina a sintaxe abstracta das novas expressões e do novo tipo, e as regras de inferência de tipo para as novas expressões.
2. Indique as novas formas canónicas da linguagem e as novas regras de avaliação *call-by-name*. (Obs.: Como responderia para *call-by-value*?)
3. Defina uma função `contaAMV`, de tipo $\text{TTree } \theta \rightarrow \text{Int}$, que conta o número “árvores do meio vazias” (i.e. que aparecem na posição intermédia dos nodos e estão vazias) que uma dada árvore tem.

Grupo 4

Considere os seguintes termos do λ -calculus:

$$A \equiv (\lambda n. \lambda x. \lambda y. n (\lambda z. y) x) \quad B \equiv (\lambda a. \lambda f. \lambda b. f (a f b)) \quad I \equiv (\lambda x. x)$$

1. Apresente a sequência correspondente à *ordem normal de redução* do termo $IA(BI)$ até à sua *forma normal*. Em cada passo de redução sublinhe o β -redex que foi selecionado.
2. Apresente a sequência de redução correspondente à avaliação *call-by-name* do termo $BI(AB)$ até à sua *forma canónica*. Em cada passo de redução sublinhe o β -redex que foi selecionado.
3. Considere a expressão A . Coloque anotações de tipo nas variáveis que estão a ser abstraídas de forma a que esta expressão seja tipificável, e indique qual o seu tipo. (**Nota:** não precisa de apresentar a prova formal do juízo de tipificação.)

Grupo 5

1. Na linguagem funcional que estudou, defina uma função que recebe um inteiro e uma lista de inteiros, e calcula quantos elementos da lista são maiores do que o inteiro recebido.
2. Apresente, passo a passo, a sequência da avaliação *call-by-name* da expressão

$$\begin{aligned} \text{let sumfst} &\equiv \lambda x. \lambda y. x + y.1, \\ \text{head} &\equiv \lambda l. \text{listcase } l \text{ of } (0, \lambda h. \lambda t. h) \\ \text{in sumfst} &(\text{head } (4+3) :: 8 :: \text{nil}) \langle 2, 3*3 \rangle \end{aligned}$$

(**Obs.:** Como responderia se a estratégia fosse *call-by-value*?)

3. Construa uma árvore de prova do juízo

$$\text{head} : \text{List Int} \rightarrow \text{Int}, a : \text{List Int} \vdash \text{let sumfst} \equiv \lambda x. \lambda y. x + y.1 \text{ in sumfst } (\text{head } a) \langle 1, 2 \rangle : \text{Int}$$

Grupo 6

Pretende-se estender a linguagem de programação funcional, com um novo tipo de dados para representar uma sequência em que os elementos podem ser acrescentados do lado esquerdo (*Esq*) ou do lado direito (*Dir*) da sequência. Por exemplo, um equivalente em Haskell desta estrutura de dados seria:

```
data Seq a = Null | Esq a (Seq a) | Dir (Seq a) a
```

1. Defina a sintaxe abstracta das novas expressões e do novo tipo, e as regras de inferência de tipo para as novas expressões.
2. Indique as novas formas canónicas da linguagem e as novas regras de avaliação *call-by-value*. (**Obs.:** Como responderia para *call-by-name*?)
3. Defina uma função que calcula o comprimento de uma sequência.